

Performance Study: Running Times and Generate Module Tests

2026-05-10

Performance Study: Running Times and generate Module Tests

1 Running Time Benchmarks

1.1 Methodology

Benchmarks were collected with **pyperf 2.10.0** using **--fast** mode (5 warmup runs, 5 measurement runs per benchmark, loop counts auto-calibrated by pyperf). Raw results are stored in **benchmarks/results.json**. **inject_random_spikes** timings were collected separately with **time.perf_counter** (20 loops, 3 warmup) because the benchmark script's background run was truncated.

Environment

Key	Value
Machine	Apple Silicon (arm64), 8 CPU cores
OS	macOS 15.7.4
Python	CPython 3.14.0 (64-bit, GIL disabled)
pyperf	2.10.0
Timer resolution	<code>mach_absolute_time()</code> , 41.7 ns

Input series for **find** and **inject** benchmarks are $A \cdot \sin(2\pi \cdot 3 \cdot x)$ sampled over $[0, 1]$, seeded at 42 for the random injectors.

1.2 Results

generate.polynomial — $O(N \cdot C)$ Fixed $C=3$ (`coefficients=(1.0, 2.0, 3.0)`):

N	Mean $\pm$ std dev	N $\times$ 10 ratio
1,000	14.87 $\pm$ 0.15 μ s	—
10,000	44.19 $\pm$ 0.19 μ s	2.97 $\times$
100,000	415.23 $\pm$ 2.73 μ s	9.39 $\times$

Effect of C at N = 10,000:

C	Mean $\pm$ std dev	C $\times$ (10/3) ratio
3	44.19 $\pm$ 0.19 μ s	—
10	103.92 $\pm$ 0.62 μ s	2.35 $\times$ (expected 3.33 $\times$)

generate.sinusoid — O(N)

N	Mean $\pm$ std dev	N $\times$ 10 ratio
1,000	14.51 $\pm$ 0.25 μ s	—
10,000	76.55 $\pm$ 3.29 μ s	5.27 $\times$
100,000	772.71 $\pm$ 13.39 μ s	10.09 $\times$

generate.exponential — O(N)

N	Mean $\pm$ std dev	N $\times$ 10 ratio
1,000	10.89 $\pm$ 0.08 μ s	—
10,000	55.83 $\pm$ 0.11 μ s	5.13 $\times$
100,000	592.04 $\pm$ 26.35 μ s	10.60 $\times$

generate.to_csv — O(N)

N	Mean $\pm$ std dev	N $\times$ 10 ratio
1,000	1,561 $\pm$ 9.9 μ s	—
10,000	14,863 $\pm$ 118 μ s	9.52 $\times$
100,000	145,450 $\pm$ 1,408 μ s	9.79 $\times$

find.detect_robust — O(N log N) theoretical

N	Mean $\pm$ std dev	N $\times$ 10 ratio
1,000	121.19 $\pm$ 1.38 μ s	—
10,000	333.04 $\pm$ 1.24 μ s	2.75 $\times$

N	Mean $\pm$ std dev	N $\times$ 10 ratio
100,000	2,978.79 $\pm$ 190.56 μ s	8.94 $\times$

inject.inject_step — $O(N)$

N	Mean $\pm$ std dev	N $\times$ 10 ratio
1,000	24.69 $\pm$ 0.14 μ s	—
10,000	256.10 $\pm$ 2.66 μ s	10.37 $\times$
100,000	2,587.84 $\pm$ 12.49 μ s	10.10 $\times$

inject.inject_random_spikes — $O(N + K)$ K grows with N in the first two rows ($K = N/100$); the third row holds K fixed:

N	K	Mean (μ s)	Ratio
1,000	10	18.50	—
10,000	100	82.02	4.43 $\times$ (N and K both $\times 10$)
100,000	100	394.58	4.81 $\times$ (N $\times 10$, K fixed)

1.3 Comparison with Theoretical Complexity

The theoretical predictions are taken from `docs/algorithms_study.md`. The table below shows predicted N $\times$ 10 growth ratios alongside what was observed at the larger (N=10k $\rightarrow$ 100k) step, where fixed per-call overheads are smallest relative to actual work.

Function	Theory	Predicted ratio (N $\times$ 10)	Observed ratio	Verdict
polynomial (C fixed)	$O(N \cdot C)$	10 $\times$	9.39 $\times$	Confirms $O(N)$; linspace dominates over polyval
sinusoid	$O(N)$	10 $\times$	10.09 $\times$	Exact
exponential	$O(N)$	10 $\times$	10.60 $\times$	Exact (within noise)
to_csv	$O(N)$	10 $\times$	9.79 $\times$	Exact; I/O-bound row loop

Function	Theory	Predicted ratio ($N \times 10$)	Observed ratio	Verdict
<code>detect_robust</code>	$O(N \log N)$	$\sim 11.3 \times$	$8.94 \times$	Consistent with $O(N) - O(N \log N)$; see below
<code>inject_step</code>	$O(N)$	$10 \times$	$10.10 \times$	Exact
<code>inject_random_spikes</code>	$O(N + K)$	depends on K	$4.81 \times$ (K fixed)	Consistent; see below

generate functions. At small N ($1k \rightarrow 10k$), all three vectorized generators show lower-than-expected ratios ($2.97 \times - 5.27 \times$). This is not a complexity anomaly — it reflects fixed NumPy overhead (array allocation, `linspace` setup, Python call stack) that is non-negligible when $N=1k$ takes only $\sim 10-15 \mu s$ total. At $N=10k \rightarrow 100k$ those costs amortize and all three generators converge to the expected $10 \times$ ratio, confirming $O(N)$.

The C effect in `polynomial` is weaker than the naive $O(C)$ factor suggests ($2.35 \times$ vs $3.33 \times$). `np.polyval` on a degree-9 polynomial at $N=10k$ is faster than a simple flop count predicts because NumPy’s Horner-scheme evaluation is branch-free and cache-hot; the `np.linspace` call (shared between both $C=3$ and $C=10$ runs) also adds a constant-fraction cost.

generate.to_csv. This is the slowest generator by two orders of magnitude (1.56 ms vs $\sim 15 \mu s$ for `polynomial` at $N=1k$), and the only I/O-bound function in the suite. The row-by-row Python CSV loop dominates; it scales nearly perfectly as $O(N)$ ($9.52 \times$ and $9.79 \times$), confirming the theoretical estimate.

find.detect_robust. The $2.75 \times$ ratio at $N=1k \rightarrow 10k$ is misleadingly low. At $N=1k$, $121 \mu s$ is consumed by `scipy.signal.find_peaks` initialization and array allocation overhead that does not grow with N . Once $N=10k$, those fixed costs become a small fraction of runtime, and the $8.94 \times$ ratio at $N=10k \rightarrow 100k$ confirms near-linear scaling. `algorithms_study.md` lists the bound as $O(N \log N)$ under the conservative assumption that `np.median` requires sort-based work; in NumPy’s implementation `np.median` uses introselect (partial sort, $O(N)$ average) so the actual median cost is linear. The observed $8.94 \times$ ratio sits squarely between $O(N)$ ($\rightarrow 10 \times$) and $O(N \log N)$ ($\rightarrow 11.3 \times$ for this range), consistent with the theoretical bound being conservative.

inject_random_spikes. When both N and K grow $10 \times$ the observed $4.43 \times$ ratio reflects the $O(N + K)$ decomposition: `list(values)` is $O(N)$ and dominates at large N , but `random.sample(range(N), K)` is $O(K)$ for $K \ll N$ and adds a fixed overhead that delays the onset of the linear regime. Holding $K=100$ fixed and growing N from $10k$ to $100k$ (the $4.81 \times$ row) shows that even with K fixed the copy cost has not yet fully dominated at $N=10k$. By $N=100k$ the copy grows

toward 10× relative to larger N values.

2 `spice_discontinuity.generate` — Tests and Results

2.1 Functions Under Test

`spice_discontinuity/generate.py` provides four functions:

Function	Signature (simplified)	What it returns
<code>polynomial</code>	<code>(n, coefficients, x_range=(0,1))</code>	N samples of $c_0 + c_1x + c_2x^2 + \dots$
<code>sinusoid</code>	<code>(n, amplitude=1, frequency=1, phase=0, x_range=(0,1))</code>	N samples of $A\sin(2\pi f x + \phi)$
<code>exponential</code>	<code>(n, rate=1, x_range=(0,1))</code>	N samples of $\exp(rate \cdot x)$
<code>to_csv</code>	<code>(path, x, y, x_col="x", y_col="y")</code>	Writes two-column UTF-8 CSV

All four are exercised by the integration test suite; `to_csv` is used as the bridge between the generator functions and the CLI detector under test.

2.2 Test Suite: `tests/test_continuous.py`

The file contains two test classes. Both classes share the same configuration file (`tests/generated_data_config.yaml`) which sets the detector parameters: `sensitivity=1.0`, `min_prominence=20.0`, `min_separation=3`, `independent_col="x"`.

Class 1 — `TestContinuousSignalsNoFalsePositives` Asserts that smooth, clean signals produce **zero detections**. The helper `_assert_clean` writes each signal to a temp CSV, runs the CLI, and checks that the output contains "No discontinuities found."

Test	Function	N	Key parameters	Assertion
<code>test_polynomial_continuous</code>	<code>polynomial</code>	200	<code>coeff=(1.0, 2.0)</code> , <code>x</code> in <code>[0,1]</code>	no detections
<code>test_polynomial_discontinuous</code>	<code>polynomial</code>	200	<code>coeff=(0,0,1)</code> , <code>x</code> in <code>[0,1]</code>	no detections

Test	Function	N	Key parameters	Assertion
test_polynomial_polynomial	polynomial	200	coeff=(0,-1,0,1), x in [-1,1]	no detections
test_sinusoid_sinusoid	sinusoid	200	A=1, f=1, phi=0	no detections
test_sinusoid_high_freq	sinusoid	500	A=1, f=10, phi=0	no detections
test_exponential_exponential	exponential	200	rate=1	no detections
test_exponential_fast	exponential	200	rate=5	no detections

Class 2 — TestInjectedDiscontinuitiesDetected Asserts that a step fault injected at the signal midpoint is **always detected**. The helper `_assert_detected` scales the step magnitude to $50 * (y_{\max} - y_{\min} + 1)$ so the injected discontinuity dwarfs any natural signal variation. It then asserts that the CLI output does *not* contain "No discontinuities found.".

Test	Function	N	Injection	Assertion
test_polynomial_polynomial_detected	polynomial	200	step at index 100	detected
test_polynomial_polynomial_detected	polynomial	200	step at index 100	detected
test_polynomial_polynomial_detected	polynomial	200	step at index 100	detected
test_sinusoid_sinusoid_detected	sinusoid	200	step at index 100	detected
test_sinusoid_high_freq_detected	sinusoid	500	step at index 250	detected
test_exponential_exponential_detected	exponential	200	step at index 100	detected
test_exponential_fast_detected	exponential	200	step at index 100	detected

2.3 Test Results

Run with `python -m pytest tests/test_continuous.py -v`:

```
===== test session starts =====
platform darwin -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/neal/Projects/uky/ee584/spice_model_discontinuity
```

```
tests/test_continuous.py::TestContinuousSignalsNoFalsePositives::test_exponential_fast PASSED
tests/test_continuous.py::TestContinuousSignalsNoFalsePositives::test_exponential_slow PASSED
```

```

tests/test_continuous.py::TestContinuousSignalsNoFalsePositives::test_polynomial_cubic PASS
tests/test_continuous.py::TestContinuousSignalsNoFalsePositives::test_polynomial_linear PASS
tests/test_continuous.py::TestContinuousSignalsNoFalsePositives::test_polynomial_quadratic PASS
tests/test_continuous.py::TestContinuousSignalsNoFalsePositives::test_sinusoid_high_freq PASS
tests/test_continuous.py::TestContinuousSignalsNoFalsePositives::test_sinusoid_low_freq PASS
tests/test_continuous.py::TestInjectedDiscontinuitiesDetected::test_exponential_fast_detected PASS
tests/test_continuous.py::TestInjectedDiscontinuitiesDetected::test_exponential_slow_detected PASS
tests/test_continuous.py::TestInjectedDiscontinuitiesDetected::test_polynomial_cubic_detected PASS
tests/test_continuous.py::TestInjectedDiscontinuitiesDetected::test_polynomial_linear_detected PASS
tests/test_continuous.py::TestInjectedDiscontinuitiesDetected::test_polynomial_quadratic_detected PASS
tests/test_continuous.py::TestInjectedDiscontinuitiesDetected::test_sinusoid_high_freq_detected PASS
tests/test_continuous.py::TestInjectedDiscontinuitiesDetected::test_sinusoid_low_freq_detected PASS

```

===== 14 passed in 1.11s =====

14 / 14 passed. All seven clean-signal tests produce zero false positives, and all seven injection tests successfully detect the injected step across all three generator types and all tested parameter combinations.